



(12) 发明专利

(10) 授权公告号 CN 112783476 B

(45) 授权公告日 2022. 02. 22

(21) 申请号 202110054596.3

CN 104375806 A, 2015.02.25

(22) 申请日 2021.01.15

CN 101957840 A, 2011.01.26

(65) 同一申请的已公布的文献号

CN 110705184 A, 2020.01.17

申请公布号 CN 112783476 A

CN 103294899 A, 2013.09.11

(43) 申请公布日 2021.05.11

CN 108875207 A, 2018.11.23

US 2013332908 A1, 2013.12.12

(73) 专利权人 中国核动力研究设计院

明平洲 等.堆芯计算的框架方法及其原型

地址 610000 四川省成都市双流区长顺大道一段328号

软件NAC4R的设计.《中国核电》.2020,第13卷(第5期),第599-605页.

(72) 发明人 明平洲 刘婷 李治刚 潘俊杰
芦韡 余红星 夏榜样 刘东
安萍

廉丽莉 等.核反应堆堆芯功率分布数值模拟.《兵器装备工程学报》.2020,第41卷(第2期),第49-53,106页.

(74) 专利代理机构 成都行之专利代理事务所
(普通合伙) 51220

Hui H.Gan 等.Broadband Spectral Numerical Green's Function for Electromagnetic Analysis of Inhomogeneous Objects.《IEEE Antennas and Wireless Propagation Letters》.2020,第19卷(第7期),第1063-1067页.

代理人 张超

(51) Int.Cl.

G06F 8/20 (2018.01)

审查员 黄琦

(56) 对比文件

CN 111198688 A, 2020.05.26

权利要求书2页 说明书8页 附图1页

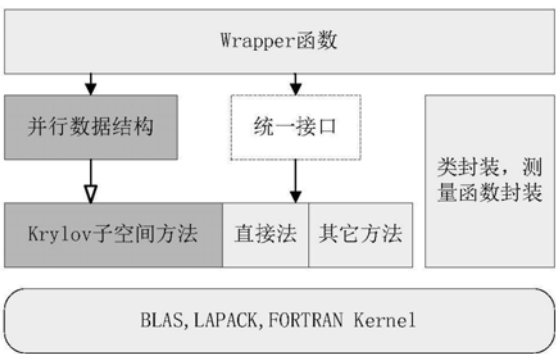
(54) 发明名称

同需求的数值求解接口。

一种堆芯数值求解器易扩展软件系统、调用方法及终端

(57) 摘要

本发明公开了一种堆芯数值求解器易扩展软件系统、调用方法及终端,涉及计算机软件技术领域,其技术方案要点是:包裹函数层提供统一的函数接口;数据层将线性系统所需数据以并行数据结构进行存储;数值方法层,将线性系统所需数据实现具体运算的数值算法进行相互独立存储;控制功能封装层提供类封装、测量函数注册以对所管理的参数列表进行测量和属性判断;结构底层存储实现数值算法的通用程序代码,并将程序代码以功能类别划分成多个核心部分进行独立储存,以及向数值方法层提供固定的函数接口。本发明能适应串行情况和分布式内存并行情况,并可提升程序模块的可扩展性和可测量性,便于集成新的数值求解算法和派生满足不



1. 一种堆芯数值求解器易扩展软件系统,其特征是,该系统应用于非线性系统的堆芯数值求解,包括:

包裹函数层,用于面向开发用户或计算用户提供统一的函数接口;

问题抽象接口层,用于对输入的参数进行抽象描述,并将非线性方程组的求解拆分为不变的几个过程以确保软件结构内计算流程从上到下有多个可选和选用不同的数值方法均能完成非线性方程组的求解,且覆盖满足串行情况和分布式内存并行情况需求的函数接口功能;

数据层,用于将非线性系统所需数据以同时适用于串行情况和分布式内存并行情况的并行数据结构进行存储,以及以适用于串行情况的串行数据结构进行存储;

数值方法层,用于将非线性系统所需数据实现具体运算的数值算法进行相互独立存储,并向包裹函数层提供统一的函数接口,以及根据函数接口传递的类型值调用对应的数值方法进行运算;

控制功能封装层,用于提供类封装、测量函数注册以对所管理的参数列表进行测量和属性判断,以及对非线性方程组求解的中间过程进行测量;

结构底层,用于存储实现数值算法的通用程序代码和共性计算内容,并将程序代码以功能类别划分成多个核心部分进行独立储存,以及向数值方法层提供固定的函数接口。

2. 根据权利要求1所述的一种堆芯数值求解器易扩展软件系统,其特征是,所述函数接口以函数形式或类形式给出。

3. 根据权利要求1所述的一种堆芯数值求解器易扩展软件系统,其特征是,所述数值方法层将非线性方程组求解的数值方法区分为串行情况和分布式内存并行情况后独立存储。

4. 根据权利要求1所述的一种堆芯数值求解器易扩展软件系统,其特征是,所述控制功能封装层为单层功能结构,向下用于统计线性方程组求解的结果,向上用于记录和更新描述非线性方程组的参数列表。

5. 一种实现如权利要求1所述的一种堆芯数值求解器易扩展软件系统的调用方法,其特征是,包括以下步骤:

S201:通过包裹函数层面向开发用户或计算用户提供统一的函数接口;

S202:通过问题抽象接口层对输入的参数进行抽象描述,并将非线性方程组的求解拆分为不变的几个过程以确保软件结构内计算流程从上到下有多个可选和选用不同的数值方法均能完成非线性方程组的求解,且覆盖满足串行情况和分布式内存并行情况需求的函数接口功能;

S203:通过数据层将非线性系统所需数据以同时适用于串行情况和分布式内存并行情况的并行数据结构进行存储,以及以适用于串行情况的串行数据结构进行存储;

S204:通过数值方法层将非线性系统所需数据实现具体运算的数值算法进行相互独立存储,并向包裹函数层提供统一的函数接口,以及根据函数接口传递的类型值调用对应的数值方法进行运算;

S205:通过控制功能封装层提供类封装、测量函数注册以对所管理的参数列表进行测量和属性判断,以及对非线性方程组求解的中间过程进行测量;

S206:通过结构底层存储实现数值算法的通用程序代码和共性计算内容,并将程序代码以功能类别划分成多个核心部分进行独立储存,以及向数值方法层提供固定的函数接

口。

6. 一种计算机终端, 包含存储器、处理器及存储在存储器并可在处理器上运行的计算机程序, 其特征是, 所述处理器执行所述程序时实现如权利要求1所述的一种堆芯数值求解器易扩展软件系统的调用方法。

一种堆芯数值求解器易扩展软件系统、调用方法及终端

技术领域

[0001] 本发明涉及计算机软件技术领域,更具体地说,它涉及一种堆芯数值求解器易扩展软件系统、调用方法及终端。

背景技术

[0002] 堆芯数值分析计算在稳态或瞬态情况下涉及求解线性系统或非线性系统,它们依赖于底层的数值求解器,例如直接法,Krylov子空间方法,Newton迭代法等。这些数值求解器可以由自研程序模块或第三方库进行提供,尽管整体算法较为固定,但编程实现细节和局部算法的改进各不相同,使得数值求解器在串行情况和分布式内存并行情况下呈现不同的计算效率和稳定性,而且会影响新算法在原有基础上实现的可扩展性。因此,如何研究设计一种堆芯数值求解器易扩展软件系统、调用方法及终端是我们目前急需解决的问题。

发明内容

[0003] 为解决现有数值求解器在串行情况和分布式内存并行情况下呈现不同的计算效率和稳定性,而且会影响新算法在原有基础上实现的可扩展性的问题,本发明的目的是提供一种堆芯数值求解器易扩展软件系统、调用方法及终端。

[0004] 本发明的上述技术目的是通过以下技术方案得以实现的:

[0005] 第一方面,提供了一种堆芯数值求解器易扩展软件系统,该系统应用于线性系统的堆芯数值求解,包括:

[0006] 包裹函数层,用于面向开发用户或计算用户提供统一的函数接口;

[0007] 数据层,用于将线性系统所需数据以同时适用于串行情况和分布式内存并行情况的并行数据结构进行存储;

[0008] 数值方法层,用于将线性系统所需数据实现具体运算的数值算法进行相互独立存储,并向包裹函数层提供统一的函数接口,以及根据函数接口传递的类型值调用对应的数值方法进行运算;

[0009] 控制功能封装层,用于提供类封装、测量函数注册以对所管理的参数列表进行测量和属性判断;

[0010] 结构底层,用于存储实现数值算法的通用程序代码,并将程序代码以功能类别划分成多个核心部分进行独立储存,以及向数值方法层提供固定的函数接口。

[0011] 进一步的,所述函数接口以函数形式或类形式给出。

[0012] 进一步的,所述控制功能封装层为跨层功能结构。

[0013] 第二方面,提供了一种实现如第一方面中所述的一种堆芯数值求解器易扩展软件系统的调用方法,包括以下步骤:

[0014] S101:通过包裹函数层面面向开发用户或计算用户提供统一的函数接口;

[0015] S102:通过数据层将线性系统所需数据以同时适用于串行情况和分布式内存并行情况的并行数据结构进行存储;

[0016] S103:通过数值方法层将线性系统所需数据实现具体运算的数值算法进行相互独立存储,并向包裹函数层提供统一的函数接口,以及根据函数接口传递的类型值调用对应的数值方法进行运算;

[0017] S104:提供控制功能封装层提供类封装、测量函数注册以对所管理的参数列表进行测量和属性判断;

[0018] S105:通过结构底层存储实现数值算法的通用程序代码,并将程序代码以功能类别划分成多个核心部分进行独立储存,以及向数值方法层提供固定的函数接口。

[0019] 第三方面,提供了一种堆芯数值求解器易扩展软件系统,该系统应用于非线性系统的堆芯数值求解,包括:

[0020] 包裹函数层,用于面向开发用户或计算用户提供统一的函数接口;

[0021] 问题抽象接口层,用于对输入的参数进行抽象描述,并将非线性方程组的求解拆分为不变的几个过程以确保软件结构内计算流程从上到下有多个可选和选用不同的数值方法均能完成非线性方程组的求解,且覆盖满足串行情况和分布式内存并行情况需求的函数接口功能;

[0022] 数据层,用于将非线性系统所需数据以同时适用于串行情况和分布式内存并行情况的并行数据结构进行存储,以及以适用于串行情况的串行数据结构进行存储;

[0023] 数值方法层,用于将非线性系统所需数据实现具体运算的数值算法进行相互独立存储,并向包裹函数层提供统一的函数接口,以及根据函数接口传递的类型值调用对应的数值方法进行运算;

[0024] 控制功能封装层,用于提供类封装、测量函数注册以对所管理的参数列表进行测量和属性判断,以及对非线性方程组求解的中间过程进行测量;

[0025] 结构底层,用于存储实现数值算法的通用程序代码和共性计算内容,并将程序代码以功能类别划分成多个核心部分进行独立储存,以及向数值方法层提供固定的函数接口。

[0026] 进一步的,所述函数接口以函数形式或类形式给出。

[0027] 进一步的,所述数值方法层将非线性方程组求解的数值方法区分为串行情况和分布式内存并行情况后独立存储。

[0028] 进一步的,所述控制功能封装层为单层功能结构,向下用于统计线性方程组求解的结果,向上用于记录和更新描述非线性方程组的参数列表。

[0029] 第四方面,提供了一种实现如第三方面所述的一种堆芯数值求解器易扩展软件系统的调用方法,包括以下步骤:

[0030] S201:通过包裹函数层面向开发用户或计算用户提供统一的函数接口;

[0031] S202:通过问题抽象接口层对输入的参数进行抽象描述,并将非线性方程组的求解拆分为不变的几个过程以确保软件结构内计算流程从上到下有多个可选和选用不同的数值方法均能完成非线性方程组的求解,且覆盖满足串行情况和分布式内存并行情况需求的函数接口功能;

[0032] S203:通过数据层将非线性系统所需数据以同时适用于串行情况和分布式内存并行情况的并行数据结构进行存储,以及以适用于串行情况的串行数据结构进行存储;

[0033] S204:通过数值方法层将非线性系统所需数据实现具体运算的数值算法进行相互

独立存储,并向包裹函数层提供统一的函数接口,以及根据函数接口传递的类型值调用对应的数值方法进行运算;

[0034] S205:通过控制功能封装层提供类封装、测量函数注册以对所管理的参数列表进行测量和属性判断,以及对非线性方程组求解的中间过程进行测量;

[0035] S206:通过结构底层存储实现数值算法的通用程序代码和共性计算内容,并将程序代码以功能类别划分成多个核心部分进行独立储存,以及向数值方法层提供固定的函数接口。

[0036] 第五方面,提供了一种计算机终端,包含存储器、处理器及存储在存储器并可在处理器上运行的计算机程序,所述处理器执行所述程序时实现如第二方面或第四方面所述的一种堆芯数值求解器易扩展软件系统的调用方法。

[0037] 与现有技术相比,本发明具有以下有益效果:

[0038] 1、本发明提供的技术方案一方面适应串行情况和分布式内存并行情况,另一方面提升程序模块的可扩展性和可测量性,便于集成新的数值求解算法和派生满足不同需求的数值求解接口;

[0039] 2、本发明提供的线性系统求解器的结构设计保证了线性方程组求解的功能模块划分不至于过细,也明确了功能模块之间的相互关联关系;相应的结构设计也具备较强的可扩展性,引入新的数值算法或测量内容仅需在结构的某个层次添加对应的模块,并不影响整个结构设计的完整性;且在保持结构内重要名称不变的前提下,修订已有的数据结构和数值算法并不会带来结构设计的改变;

[0040] 3、本发明提供的非线性系统求解器的结构设计在底层复用线性系统求解器的软件结构,同时在中间层次引入问题抽象接口来降低非线性方程组问题的描述复杂度;此结构设计具备较强的可扩展性,可以描述Jacobi矩阵已知或未知的情况,也便于引入新的数值算法和求解流程;整个结构设计内问题抽象接口保持不变后,修订已有的数据结构和数值算法并不会带来结构设计的改变。

附图说明

[0041] 此处所说明的附图用来提供对本发明实施例的进一步理解,构成本申请的一部分,并不构成对本发明实施例的限定。在附图中:

[0042] 图1是本发明实施例中求解线性系统的系统框图;

[0043] 图2是本发明实施例中求解非线性系统的系统框图。

具体实施方式

[0044] 为使本发明的目的、技术方案和优点更加清楚明白,下面结合实施例和附图,对本发明作进一步的详细说明,本发明的示意性实施方式及其说明仅用于解释本发明,并不作为对本发明的限定。

[0045] 实施例1

[0046] 一种堆芯数值求解器易扩展软件系统,如图1所示,该系统应用于线性系统的堆芯数值求解,该系统包裹函数层、数据层、数值方法层、控制功能封装层、结构底层。

[0047] (1) 包裹函数层(Wrapper)面向开发用户或计算用户提供统一的函数接口,函数接

口以函数形式或类形式给出，它符合堆芯软件研发团队的理解习惯和使用习惯，可以通过C++编程语言提供的类及其封装、继承等机制加以实现。例如一种串行的包裹函数接口形式为：void bicg_solver_serial(double*A,double*b,double*x) {}。以双向稳定共轭梯度求解器为例，线性系统求解器的形式化示例如表1所示：

[0048] 表1线性系统求解器的形式化示例

	函数形式	类形式
[0049]	void Bicg_solver0(int64_t n, CuComputeMatrix &A, double64_t *x, double64_t *b, double64_t epsilon) { }	class solver0 { private: int64_t n; CuCrsMatrix A; double64_t *x; double64_t *b; double64_t epsilon; public:
	void Bicg_mpi_solver0(int64_t n, CuCrsMatrix &A, double64_t *x, double64_t *b, double64_t epsilon) { }	void Bicg_solver(); void Bicg_mpi_solver(); };
[0050]		

[0051] (2) 数据层，用于将线性系统所需数据以同时适用于串行情况和分布式内存并行情况的并行数据结构进行存储。结构的具体功能可以划分为多个层次，其中数据层提供对线性系统所需的矩阵、解向量、右手向量等数据的封装。如图1所示，并行数据结构结构表明Krylov子空间数值方法支持分布式内存环境，因此包裹函数层传递的数据，例如矩阵系数以及右手向量，将在该结构被转换为分布式内存环境上的存储形式，在每个处理器核心上依然调用底层的基础线性代数操作来完成数值运算。

[0052] 并行数据结构的引入使得求解器在使用Krylov子空间数值方法可以适应分布式内存环境，但并不会带来接口形式的改变，其中Krylov子空间方法、直接法和其它方法(新的算法或第三方函数库)均统一接口，与并行数据结构内的格式相互匹配。

[0053] (3) 数值方法层，用于将线性系统所需数据实现具体运算的数值算法进行相互独立存储，并向包裹函数层提供统一的函数接口，以及根据函数接口传递的类型值调用对应的数值方法进行运算，例如Krylov子空间方法、并行LU分解直接方法、特殊矩阵(诸如三对角矩阵)求解方法等。

[0054] (4) 控制功能封装层，用于提供类封装、测量函数注册以对所管理的参数列表进行测量和属性判断。指代的是用于对软件内部的行为进行测量的一些内容，它们可能涉及计算时间的测量，分布式内存通信次数和求解中间过程的记录，所以在结构框图中体现为跨

层的特性。

[0055] (5) 结构底层,用于存储实现数值算法的通用程序代码,并将程序代码以功能类别划分成多个核心部分进行独立储存,以及向数值方法层提供固定的函数接口。核心部分是具体的矩阵和向量的运算,可以使用已有的BLAS、LAPACK第三方库,也可以使用经过定制优化后的通用程序代码等。

[0056] 实施例2

[0057] 一种堆芯数值求解器易扩展软件系统,如图2所示,该系统应用于非线性系统的堆芯数值求解,本实施例中的软件系统基于应用于非线性系统的堆芯数值求解的软件系统进行设计的。对非线性系统(非线性方程组)求解器的软件结构与线性系统求解器较为相似,主要特点在于采用分层结构和复用线性系统求解器的软件结构。这一方面由于非线性方程组求解器底层需要依赖于线性方程组求解器,另一方面非线性方程组对于问题的描述相对复杂,采用分层结构可以减小编程实现难度。

[0058] 非线性系统求解器的软件结构需要建立在线性系统求解器基础上,这是由于主要的非线性方程组求解算法Broyden法、Newton法在数值计算过程中均需要转换为线性系统进行求解。

[0059] 该系统包括包裹函数层、问题抽象接口层、数据层、数值方法层、控制功能封装层、结构底层。

[0060] (1) 非线性系统求解器的软件结构顶层仍然是包裹函数层,面向用户提供易用的适合于堆芯研发团队使用的接口,可以用函数形式或者类形式给出,例如C++编程语言实现之后以类的形式,C编程语言实现之后以函数的形式。如表2所示:

[0061] 表2非线性系统求解器的形式化示例

	函数形式	类形式
[0062]	<pre>void newton_NLSolver_classical(int64_t n, Nox_NLProblem *problem, double64_t *x, double64_t epsilon) void newton_NLSolver_matrixfree(int64_t n, Nox_NLProblem *problem, double64_t *x, double64_t epsilon)</pre>	<pre>class Nox_NLProblem { public: Nox_NLProblem() {} virtual ~Nox_NLProblem() {} virtual void ComputeF() = 0; virtual void UpdateJacobian() = 0; Epetra_CrsMatrix * GetMatrix() { return Matrix_; } private: int64_t n; Epetra_CrsMatrix * Matrix_; };</pre>

[0063] 从示例中可以发现非线性系统求解器需要对待求解问题进行一些描述,区别于线性系统的矩阵信息在非线性系统求解器的软件结构中需要进行额外考虑和设计,例如使用Newton法时系数矩阵和Jacobi矩阵的表达更为复杂。为了减小描述难度,提出的分层软件结构引入问题抽象接口这一层次,使用虚基类等机制来限定非线性系统对系数矩阵、Jacobi矩阵、中间计算结果的表达。与此同时,不同的非线性方程组求解数值方法区分为串行情况和分布式内存并行情况,位于问题抽象接口层的下一层次。

[0064] 该结构的底层复用线性系统求解器的部分结构,同时引入其它矩阵或向量的计算核心,在此基础上引入参数列表的测量以及封装层,对非线性方程组求解过程的中间过程进行测量和记录,且独立于复用的线性系统求解器内部的测量函数。

[0065] (2) 问题抽象接口层,用于对输入的参数进行抽象描述,并将非线性方程组的求解拆分为不变的几个过程以确保软件结构内计算流程从上到下有多个可选和选用不同的数值方法均能完成非线性方程组的求解,且覆盖满足串行情况和分布式内存并行情况需求的函数接口功能。例如,问题的表示、问题的转换。常规实现就是对应于串行情况,并行实现则是对应分布式内存环境,与线性方程组求解器的设计类似,将数据在并行环境下的转换隐藏在并行实现这个结构内自动实现。

[0066] (3) 数据层,用于将非线性系统所需数据以同时适用于串行情况和分布式内存并行情况的并行数据结构进行存储,以及以适用于串行情况的串行数据结构进行存储。

[0067] (4) 数值方法层,用于将非线性系统所需数据实现具体运算的数值算法进行相互

独立存储,并向包裹函数层提供统一的函数接口,以及根据函数接口传递的类型值调用对应的数值方法进行运算。

[0068] (5)控制功能封装层,用于提供类封装、测量函数注册以对所管理的参数列表进行测量和属性判断,以及对非线性方程组求解的中间过程进行测量。

[0069] (6)结构底层,用于存储实现数值算法的通用程序代码和共性计算内容,并将程序代码以功能类别划分成多个核心部分进行独立储存,以及向数值方法层提供固定的函数接口。其它计算核心是需要放于整个结构底层的一些共性计算内容,例如非线性方程组的表示转换等操作。

[0070] 在具体实施过程中,可以有如下的实施机制:

[0071] 一、层次结构的考虑

[0072] 提出的堆芯数值求解器软件结构采用分层和跨层的设计,在编程实现时相同层次结构的内容可以相互独立或者存在依赖关系。相互独立强调某个计算功能在调用层次中位置不同,跨层则强调某些计算功能可以绕开一些不必要的调用内容,这样使得提出的软件结构较为灵活。由于线性系统和非线性系统的单向依赖关系,这里考虑层次结构的实现时,线性系统求解器和非线性系统求解器应相互独立。由于串行情况和分布式内存并行情况在计算过程中具备部分功能的重叠性,所以串行情况和分布式内存并行情况下计算流程应采用跨层设计,并将公共的功能部分作为提出的软件结构的底层(例如围绕矩阵和向量的操作)。

[0073] 二、函数接口的固定

[0074] 每种求解器内部实现时可以采用抽象类或抽象函数接口来进行接口的固定,这使得提出的软件结构在同一层次具备较强的灵活性,如表3所示:

[0075] 表3固定函数接口的示例

[0076]	<pre>class Base { public: virtual void f() = 0; virtual void g() = 0; }</pre>	<pre>class M1 : public Base { public: void f(); void g(); }</pre>
		<pre>class M2 : public Base { public: void f(); void g(); }</pre>

[0077] 表3中M1和M2内的成员函数具有固定的函数接口,但内部实现可以不同,同时利用

C++编程语言的封装、多态等特性,引申至更为常见的情况。

[0078] 三、可扩展性的考虑

[0079] 提出的软件结构应具备灵活的可扩展性,一方面可以引入面向对象中的设计模式,例如工厂模式、代理模式和适配器模式等内容,增强软件结构在每个层次上的横向可扩展;另一方面可以引入回调函数和预留接口的方法,增强软件结构的纵向可扩展。如表4所示:

[0080] 表4扩展性的示例

工厂模式	回调函数
[0081] <pre>class S_Factory { public: S build(); S build_other(); }</pre>	<pre>void reg(S s_outer); void unreg(S s_inner); for s in list of S: ::Lamda(s) end for</pre>

[0082] 表4中工厂模式提供的横向可扩展特性除开已有算法的实现使用build方法创建结构S之外,build_other方法表明可以使用第三方函数库进行结构S的创建。回调函数则通过注册机制来对计算流程进行预留接口,使得跨层设计纵向的扩展性较为灵活。

[0083] 四、可测量性的考虑

[0084] 提出的软件结构具备测量堆芯数值求解过程,这里涉及到较多参数的更新和中间变量的使用,在分布式内存并行情况下还需要考虑各个进程上的数据变化和数据通信。软件结构中线性系统求解器和非线性求解器分别具备测量模块,其中线性系统求解器的测量模块采用跨层设计,便于多个层次结构的测量;非线性系统求解器底层依赖于线性系统求解器,采用单层设计,向下用于统计线性方程组求解的结果,向上则用于记录和更新描述非线性方程组的参数列表。

[0085] 以上所述的具体实施方式,对本发明的目的、技术方案和有益效果进行了进一步详细说明,所应理解的是,以上所述仅为本发明的具体实施方式而已,并不用于限定本发明的保护范围,凡在本发明的精神和原则之内,所做的任何修改、等同替换、改进等,均应包含在本发明的保护范围之内。

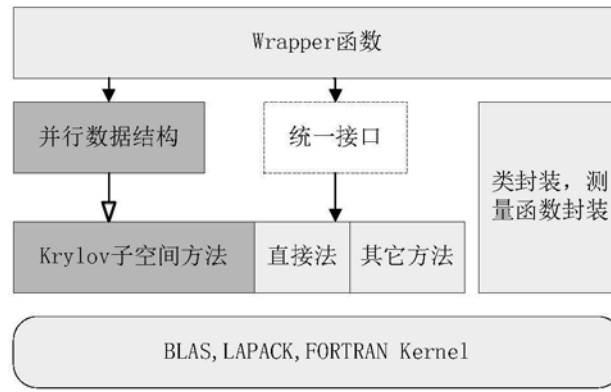


图1

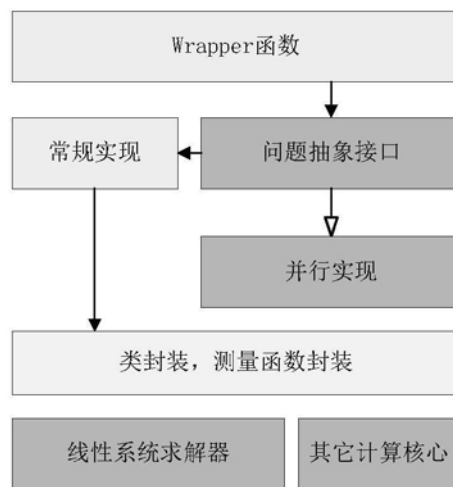


图2